LAB 6

Code and Branch Coverage Analysis

Repository	https://github.com/ashlesh-rv/SE-LABS-PES1UG24CS088.git
Project	Vending Machine
Test framework	Python 3.12.7, pytest 9.2.0, coverage 7.17.0
Code coverage tests	13 tests passed | 100% total coverage
Branch coverage tests	14 tests passed | 100% total coverage

Submission contents

1. Code coverage test file and final terminal report screenshot.
2. Branch coverage test file and final terminal report screenshot.

The coverage screenshots show the final test runs and reports for the vending machine application. The Python test files are reproduced in this report for review.

Part A - Code Coverage Test Cases

CodeCoverageTest_xxxx_application.py | 13 tests | Targets at least 90% code coverage; achieved 100% in the submitted run.

```
import pytest
from vending_machine import select_item, make_change, purchase


# SELECT ITEM TEST CASES

def test_invalid_item():
    assert select_item("water", {}, 100) == "invalid"


def test_sold_out_item():
    assert select_item("cola", {"cola": 0}, 200) == "sold_out"


def test_insufficient_credit():
    assert select_item("cola", {"cola": 5}, 100) == "insufficient"


def test_valid_item():
    assert select_item("cola", {"cola": 5}, 150) == "ok"


# MAKE CHANGE TEST CASES

def test_no_change_needed():
    assert make_change(0) == []


def test_negative_amount():
    with pytest.raises(ValueError):
        make_change(-10)


def test_exact_change():
    assert make_change(185) == [100, 50, 25, 10]


def test_no_exact_change():
    assert make_change(3) is None


# PURCHASE TEST CASES

def test_purchase_invalid_item():
    assert purchase("water", {}, 200) == ("invalid", [])


def test_purchase_sold_out():
    assert purchase("cola", {"cola": 0}, 200) == ("sold_out", [])

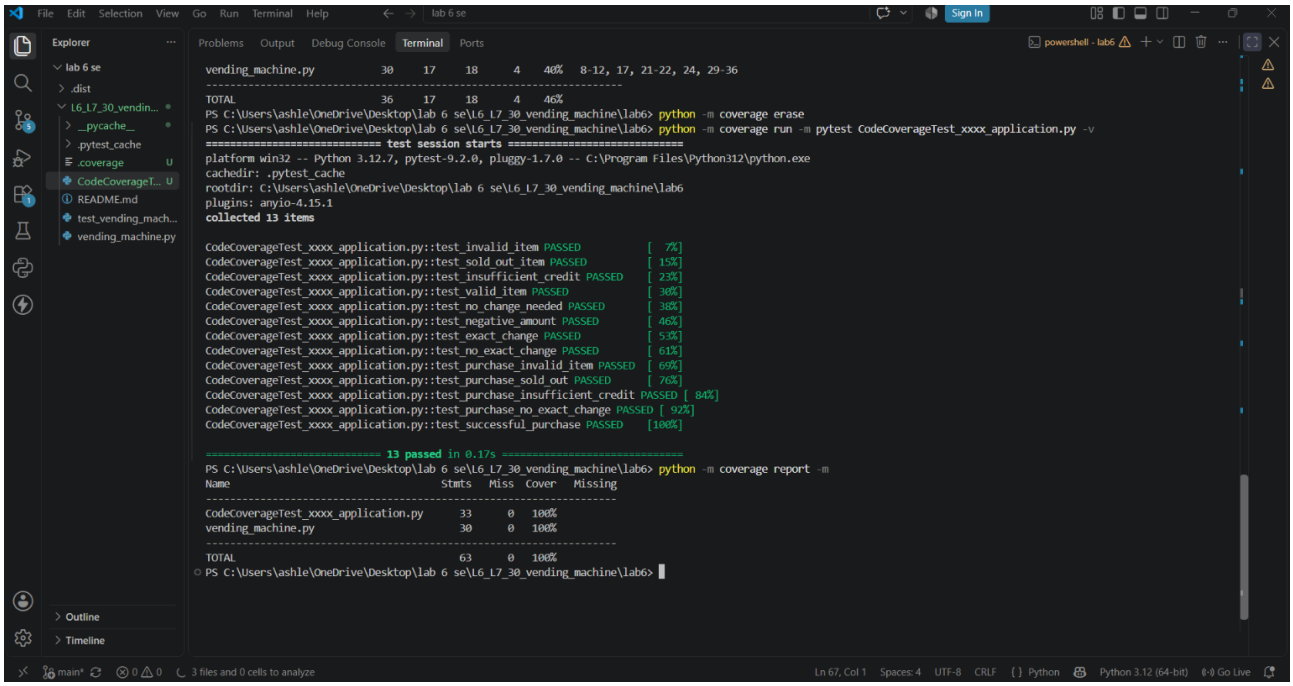

def test_purchase_insufficient_credit():
    assert purchase("cola", {"cola": 2}, 100) == ("insufficient", [])


def test_purchase_no_exact_change():
    stock = {"cola": 2}
    assert purchase("cola", stock, 153) == ("no_exact_change", [])
    assert stock["cola"] == 2


def test_successful_purchase():
    stock = {"cola": 2}
    assert purchase("cola", stock, 200) == ("dispensed", [50])
    assert stock["cola"] == 1
```

Part A - Code Coverage Results

Final terminal output: 13 tests passed; the vending_machine.py file and total coverage are both reported at 100%.



Source: terminal screenshot provided for the code coverage run.

Part B - Branch Coverage Test Cases

BranchCoverageTest_xxxx_application.py | 14 tests | Includes an additional smallest-coin test to exercise the five-unit coin path.

```
import pytest
from vending_machine import select_item, make_change, purchase


# SELECT ITEM TEST CASES

def test_invalid_item():
    assert select_item("water", {}, 100) == "invalid"


def test_sold_out_item():
    assert select_item("cola", {"cola": 0}, 200) == "sold_out"


def test_insufficient_credit():
    assert select_item("cola", {"cola": 5}, 100) == "insufficient"


def test_valid_item():
    assert select_item("cola", {"cola": 5}, 150) == "ok"


# MAKE CHANGE TEST CASES

def test_no_change_needed():
    assert make_change(0) == []


def test_negative_amount():
    with pytest.raises(ValueError):
        make_change(-10)


def test_exact_change():
    assert make_change(185) == [100, 50, 25, 10]


def test_no_exact_change():
    assert make_change(3) is None


def test_smallest_coin():
    assert make_change(5) == [5]


# PURCHASE TEST CASES

def test_purchase_invalid_item():
    assert purchase("water", {}, 200) == ("invalid", [])


def test_purchase_sold_out():
    assert purchase("cola", {"cola": 0}, 200) == ("sold_out", [])

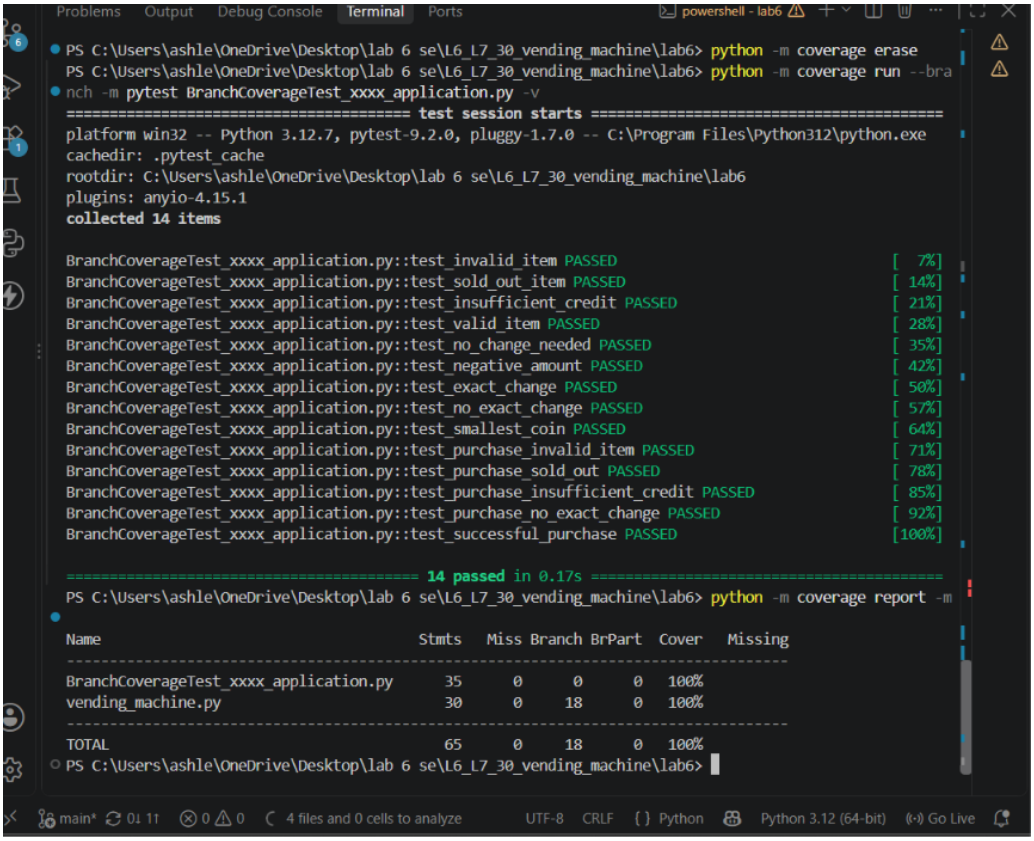

def test_purchase_insufficient_credit():
    assert purchase("cola", {"cola": 2}, 100) == ("insufficient", [])


def test_purchase_no_exact_change():
    stock = {"cola": 2}
    assert purchase("cola", stock, 153) == ("no_exact_change", [])
    assert stock["cola"] == 2


def test_successful_purchase():
    stock = {"cola": 2}
    assert purchase("cola", stock, 200) == ("dispensed", [50])
    assert stock["cola"] == 1
```

Part B - Branch Coverage Results

Final terminal output: 14 tests passed. The report shows 18 branches, 0 partial branches (BrPart), and 100% total coverage.



Source: terminal screenshot provided for the branch coverage run.